

ДИСЦИПЛИНА	Технологическая (проектно-технологическая) практика
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практика в формате НИР – “Исследование по теме ВКР”
ПРЕПОДАВАТЕЛЬ	Литвиненко Эдуард Константинович
СЕМЕСТР	1 семестр, 2025/2026 уч. год

СОДЕРЖАНИЕ ОТЧЁТА

Отчёт по практике в формате научно-исследовательской работы (НИР) представляет из себя теоретический обзор и последующее формирование требований для программного решения, технологии и инструментальным средствам для решения некоторой проблемы из ИТ-отрасли. Отрасль и проблема выбираются строго исходя из потенциальной темы выпускной квалификационной работы (ВКР). Таким образом, подготовка данного отчёта по НИР приведёт к формированию навыков анализа предметной области, теоретического обзора проблемы и постановки гипотезы/предложения технологии решения. Минимальный объём печатного текста от введения до заключения: 30 страниц.

Структура отчёта в формате НИР следующая:

ВВЕДЕНИЕ

1. ОБЗОР ТЕКУЩЕГО СОСТОЯНИЯ ПРЕДМЕТА ИССЛЕДОВАНИЯ

1.1 Обзор зарубежных исследований

1.2 Обзор отечественных исследований

1.3 Общая характеристика проблемы исследования

2. ОБЗОР ТЕХНОЛОГИЙ И МЕТОДОВ РЕШЕНИЯ ПРОБЛЕМЫ

2.1 Основные методы решения проблемы и их характеристики

2.2 Основные технологии решения проблемы и их характеристики

2.3 Алгоритм выбора методов и технологий решения проблемы

3. ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К РЕШЕНИЮ ПРОБЛЕМЫ

3.1 Общая характеристика предлагаемого решения

3.2 Функциональные требования к предлагаемому решению

3.3 Нефункциональные требования к предлагаемому решению

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Далее представлен пример приблизительной структуры отчёта. Некоторые фрагменты и структуру примера можно использовать в своём исследовании.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ ОТЧЁТА	2
СОДЕРЖАНИЕ.....	3
ВВЕДЕНИЕ	4
1. ОБЗОР ТЕКУЩЕГО СОСТОЯНИЯ ПРЕДМЕТА ИССЛЕДОВАНИЯ	8
1.1 Обзор зарубежных исследований	8
1.2 Обзор отечественных исследований	11
1.3 Общая характеристика проблемы исследования	11
2. ОБЗОР ТЕХНОЛОГИЙ И МЕТОДОВ РЕШЕНИЯ ПРОБЛЕМЫ	18
2.1 Основные методы решения проблемы и их характеристики	18
2.2 Основные технологии решения проблемы и их характеристики	18
2.3 Алгоритм выбора методов и технологий решения проблемы	24
3 ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К РЕШЕНИЮ ПРОБЛЕМЫ.....	24
3.1 Общая характеристика предлагаемого решения	28
3.2 Функциональные требования к предлагаемому решению	30
3.3 Нефункциональные требования к предлагаемому решению.....	34
ЗАКЛЮЧЕНИЕ.....	34
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	40

ВВЕДЕНИЕ

Актуальность настоящего исследования обусловлена стремительным развитием технологий потоковой передачи мультимедийного контента и ростом потребительского интереса к легальным онлайн-кинотеатрам и стриминговым сервисам. За последние годы потребительская модель потребления видео изменилась: зрители всё чаще отказываются от традиционного телевидения и физических носителей в пользу удобных онлайн-платформ, доступных с различных устройств — персональных компьютеров, смартфонов, планшетов, смарт-телевизоров и игровых приставок. При этом пользователи ожидают не только большого разнообразия контента, но и высокого качества изображения, стабильной работы сервиса и удобного интерфейса.

Наряду с современными фильмами и сериалами, значительную ценность для зрителей представляет ретроконтент — классические фильмы и аниме, выпущенные десятилетия назад и зачастую доступные лишь в низком разрешении и с артефактами исходной плёнки или ранних цифровых форматов. Для определённых аудиторий такие произведения обладают не только развлекательной, но и культурно-исторической и ностальгической ценностью. Однако техническое качество исходного материала зачастую не соответствует ожиданиям современных пользователей, привыкших к чёткой, детализированной картинке и насыщенным цветам на больших 4K-экранах.

Параллельно наблюдается широкое внедрение дисплеев с высоким разрешением (4K и выше), игровых консолей и медиаприставок, поддерживающих воспроизведение видео в форматах Ultra HD и HDR. Это приводит к заметному визуальному несоответствию между возможностями современного оборудования и качеством исходного ретровидео. Пользователь, открывая старый фильм на большом 4K-экране, сталкивается с «замыленной» картинкой, шумами, ступенчатыми контурами, артефактами сжатия и искажениями, обусловленными ограничениями исходной плёнки или ранних цифровых кодеков. Всё это снижает пользовательский опыт, а иногда делает

просмотр откровенно некомфортным, что ограничивает востребованность классических произведений у новой аудитории.

Дополнительной проблемой является то, что ретровидео часто хранится в неоднородном качестве: часть материалов оцифрована с плёнки, часть — с VHS-кассет, часть — с уже сжатых цифровых источников. Разрешение таких записей может существенно отличаться, а наличие артефактов и шумов усложняет использование банальных методов масштабирования. В результате прямое воспроизведение такого контента на современных дисплеях не раскрывает его потенциал и зачастую формирует у зрителя субъективно негативное отношение к старым произведениям, хотя их художественная ценность остаётся высокой.

Одним из перспективных направлений решения данной проблемы выступает использование методов интеллектуального масштабирования (upscaling) на основе глубоких нейросетей, в частности открытой модели Real-ESRGAN, позволяющей восстанавливать детали и повышать разрешение видеоконтента без существенной потери визуального качества. В отличие от классических алгоритмов интерполяции, нейросетевые модели обучаются на больших наборах данных и способны «додумывать» недостающие детали, восстанавливая текстуры, контуры и мелкие элементы изображения. Модель Real-ESRGAN реализована с использованием фреймворка глубокого обучения PyTorch, что обеспечивает удобство её дообучения, развёртывания и интеграции в пользовательские конвейеры обработки изображений и видео.

Однако сам по себе факт существования эффективных моделей суперразрешения ещё не решает всех задач. Необходимо создать инфраструктуру, которая позволит интегрировать подобную модель в реальный стриминговый сервис: организовать хранение больших объёмов исходного и обработанного видеоконтента, обеспечить устойчивую работу веб-интерфейса, реализовать адаптивную потоковую передачу данных, а также выстроить асинхронный конвейер обработки, способный масштабироваться при росте нагрузки. Для этих целей в настоящей работе рассматривается стек технологий, включающий язык программирования Golang, веб-фреймворк Gin, реляционную

СУБД PostgreSQL, объектное хранилище MinIO, фреймворк PyTorch и систему обмена сообщениями Kafka.

Целью данной работы является проведение комплексного анализа проблемной области, связанной с разработкой специализированного стримингового сервиса для показа ретрофильмов и ретроаниме с возможностью потоковой передачи видео в разрешении до 4K с применением модели Real-ESRGAN для предварительного или динамического upscaling. На основе проведенного анализа предполагается сформировать облик перспективного программного решения, включающего архитектуру сервиса, используемые технологии и требования к качеству предоставляемых услуг. Особое внимание уделяется вопросам практической интеграции нейросетевой обработки в архитектуру веб-сервиса, построенного на опоре микросервисного подхода и событийно-ориентированного взаимодействия.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) изучить текущее состояние исследований в области потоковой передачи видеоконтента, адаптивного стриминга и нейросетевого upscaling в зарубежных и отечественных источниках;
- 2) выполнить сравнительный анализ существующих методов организации видеостриминга и подходов к масштабированию ретровидео до 4K, выявить их преимущества и ограничения в контексте поставленной задачи;
- 3) рассмотреть стек технологий, включающий язык программирования Golang, систему управления базами данных PostgreSQL, объектное хранилище MinIO, фреймворк PyTorch, веб-фреймворк Gin и систему обмена сообщениями Kafka, с точки зрения их применимости, совместимости и перспектив масштабирования при разработке стримингового сервиса;
- 4) сформулировать функциональные и нефункциональные требования к программному комплексу, обеспечивающему потоковую передачу

ретроконтента в высоком разрешении и удобное взаимодействие пользователя с системой;

- 5) определить алгоритм выбора методов и технологий, обеспечивающий баланс между качеством изображения, производительностью, ресурсными ограничениями и удобством эксплуатации, а также учесть вопросы сопровождаемости и расширяемости решения;
- 6) описать возможную архитектуру асинхронного конвейера обработки видеоконтента, в котором Kafka используется для обмена событиями между сервисами, а PyTorch-сервис с моделью Real-ESRGAN отвечает за нейросетевой upscaling, и показать место этого конвейера в общей структуре системы.

Практическая значимость работы заключается в создании теоретического фундамента для последующей реализации стримингового веб-сервиса на базе Golang, PostgreSQL, MinIO, PyTorch, Gin router и Kafka с интеграцией модели Real-ESRGAN в рамках выпускной квалификационной работы. Полученные результаты могут быть использованы при проектировании и разработке прототипа платформы для легального показа ретрофильмов и аниме с улучшенным визуальным качеством, удобным веб-интерфейсом и расширяемой архитектурой серверной части. Кроме того, сформулированные подходы и архитектурные решения могут быть адаптированы и для других задач, связанных с обработкой и доставкой видеоконтента, в том числе в образовательных, архивных и корпоративных системах.

1. ОБЗОР ТЕКУЩЕГО СОСТОЯНИЯ ПРЕДМЕТА ИССЛЕДОВАНИЯ

1.1 Обзор зарубежных исследований

Анализ зарубежных научных публикаций и технической документации свидетельствует о значительном внимании научного и инженерного сообществ к проблематике потоковой передачи видеоконтента, адаптивного стриминга, а также улучшения качества изображения с помощью методов глубокого обучения. На стыке распределённых систем, компьютерных сетей и компьютерного зрения сформировалась устойчивая междисциплинарная область исследований, в рамках которой рассматриваются задачи, непосредственно связанные с разработкой стриминговых сервисов нового поколения.

Одним из ключевых направлений зарубежных исследований являются методы адаптивной потоковой передачи (Adaptive Bitrate Streaming, ABR). В работах, посвящённых стандартам HLS и MPEG-DASH, подробно анализируются схемы сегментирования видеопотока, механизмы формирования плейлистов и алгоритмы выбора качества воспроизведения на стороне клиента. Зарубежные авторы предлагают различные варианты ABR-алгоритмов: от простейших подходов, основанных на оценке текущей пропускной способности канала и размера буфера, до методов, использующих элементы машинного обучения и прогнозирования сетевых условий. Отдельное внимание уделяется устойчивости алгоритмов к резким скачкам пропускной способности и их способности предотвращать частую смену качества, негативно влияющую на восприятие пользователя.

Другим важным направлением зарубежных исследований является суперразрешение изображений и видео (super-resolution). В многочисленных публикациях рассматриваются как классические сверточные архитектуры, так и генеративно-состязательные сети (GAN), позволяющие получать визуально более выразительные результаты. Модель Real-ESRGAN и родственные ей

архитектуры анализируются как пример практико-ориентированного решения, которое можно использовать не только в лабораторных условиях, но и в реальных продуктах. Работы, посвящённые Real-ESRGAN, акцентируют внимание на способности модели работать с «реальными» деградированными данными, а не только с искусственно созданными парами низко- и высококачественных изображений. Это особенно важно в контексте ретровидео, где артефакты деградации могут быть крайне разнообразны.

В зарубежной литературе также рассматриваются новые подходы к обучению моделей суперразрешения, включающие использование перцепционных метрик качества, таких как SSIM и LPIPS, а также комбинированные функции потерь, учитывающие как пиксельную точность, так и визуальное восприятие наблюдателя. Исследователи экспериментируют с различными архитектурами генератора и дискриминатора, применяют техники регуляризации и оптимизации, стремясь достичь баланса между качеством результата и вычислительной сложностью модели. В ряде работ рассматривается вопрос переноса обучения (transfer learning) и дообучения моделей на специализированных наборах данных, например, содержащих кадры из старых фильмов и аниме.

Существенную часть зарубежных исследований составляют работы, посвящённые интеграции моделей компьютерного зрения в промышленные стриминговые платформы. Анализируются архитектуры, в которых суперразрешение и другие виды обработки (шумоподавление, цветокоррекция, восстановление кадров) выступают в роли отдельных сервисов, взаимодействующих с основным приложением через очереди сообщений или API. В таких системах широко применяются брокеры сообщений (Kafka, RabbitMQ и др.), обеспечивающие асинхронный обмен задачами и результатами между компонентами. Это позволяет разграничить зоны ответственности: стриминговый сервис обрабатывает пользовательские запросы, а подсистема обработки видео занимается ресурсоёмкими операциями, не блокируя основной поток запросов.

Отдельный пласт зарубежных исследований касается масштабирования и отказоустойчивости стриминговых систем. Изучаются вопросы горизонтального масштабирования серверов приложений, балансировки нагрузки между узлами, использования CDN и edge-узлов для приближения контента к конечному пользователю. В этом контексте рассматриваются микросервисные архитектуры, позволяющие независимым компонентам (каталог, авторизация, стриминг, аналитика, ML-обработка) масштабироваться по отдельности. Подробно изучаются подходы к организации мониторинга и логирования, включая сбор метрик, трассировку запросов и визуализацию состояния системы в реальном времени.

Не менее значимой темой зарубежных исследований является выбор фреймворков глубокого обучения, в частности PyTorch и TensorFlow. PyTorch рассматривается как удобная платформа для быстрой прототипизации и внедрения моделей компьютерного зрения в промышленные решения. В работах обсуждаются вопросы упаковки моделей в контейнеры, использования ONNX для переноса моделей на другие платформы, а также оптимизации вывода (inference) с помощью библиотек, поддерживающих ускорение на GPU. Всё это формирует методологическую базу для интеграции Real-ESRGAN в конвейер обработки видеоконтента стримингового сервиса.

Наконец, в зарубежных публикациях уделяется внимание пользовательскому опыту (User Experience) и восприятию качества видео. Проводятся эксперименты, в рамках которых участникам предлагается оценить субъективное качество изображения при разных сочетаниях разрешения, битрейта и методов обработки. Результаты таких исследований показывают, что грамотное применение суперразрешения и постобработки может заметно повысить субъективно воспринимаемое качество при относительно умеренных требованиях к битрейту. Это косвенно подтверждает актуальность использования подобных подходов в нишевых сервисах ретроконента.

В совокупности зарубежные исследования формируют богатую теоретическую и практическую базу, на которую можно опираться при

проектировании стримингового сервиса ретрофильмов и ретроаниме с поддержкой 4K-upscaling. Они задают ориентиры по архитектуре, методам обработки и оценке качества, а также демонстрируют жизнеспособность решений, сочетающих стриминг и нейросетевую обработку контента.

1.2 Обзор отечественных исследований

В рамках отечественной научной школы ведётся активная работа по направлениям потоковой передачи видео, построения веб-ориентированных мультимедийных сервисов и применению методов искусственного интеллекта для обработки изображений и видео. Российские исследовательские коллективы, вузы и промышленные лаборатории рассматривают как фундаментальные аспекты, так и сугубо практические задачи, связанные с созданием и эксплуатацией онлайн-платформ доставки медиаконтента.

Существенная часть отечественных исследований в области стриминга ориентирована на анализ особенностей национальной сетевой инфраструктуры и учёт требований законодательства. В публикациях поднимаются вопросы локализации контента, соблюдения авторских прав и договорных отношений с правообладателями, а также защита персональных данных пользователей. Описываются модели лицензирования и варианты организации платного доступа к контенту (подписки, аренда, рекламные модели), что особенно важно для коммерческих сервисов. Хотя в рамках выпускной квалификационной работы, как правило, реализуется учебный прототип без полной юридической инфраструктуры, понимание этих аспектов помогает формулировать требования к системе с перспективой дальнейшего развития.

Отдельное направление отечественных исследований связано с задачами импортозамещения и использования открытого программного обеспечения. Рассматриваются архитектуры онлайн-кинотеатров и медиасервисов, построенных на базе отечественных и свободных компонентов: СУБД, систем

хранения, брокеров сообщений, фреймворков для веб-разработки. В этом контексте активно обсуждаются возможности использования PostgreSQL и объектных хранилищ, совместимых с S3-интерфейсом, что созвучно выбору MinIO в проектируемой системе. Анализируются способы построения отказоустойчивых кластеров, организации резервного копирования и обеспечения непрерывной доступности сервиса при сбоях отдельных узлов.

Существенный блок отечественных работ посвящён применению методов компьютерного зрения и глубокого обучения для обработки изображений и видео. Рассматриваются задачи суперразрешения, шумоподавления, восстановления деталей, цветокоррекции и стабилизации видеоряда. В ряде публикаций исследуются особенности обработки архивного видеоматериала, оцифрованной киноплёнки и телетрансляций прошлых лет. Показано, что применение нейросетевых методов позволяет существенно повышать визуальное качество таких записей, уменьшать заметность шумов и артефактов, а также улучшать резкость и читаемость деталей.

Особое внимание в отечественных работах уделяется обработке анимации, включая аниме и другие стилизованные формы искусства. Отмечается, что такие данные имеют специфическую структуру: резкие контуры, плоские заливки, ограниченные палитры, выраженные стилевые элементы. В связи с этим стандартные модели, обученные преимущественно на фотографических изображениях, могут давать нежелательные артефакты, такие как «замыливание» контуров или искажение рисунка. В публикациях предлагаются подходы к дообучению моделей на специализированных датасетах, содержащих аниме-кадры, и к настройке функций потерь с учётом особенностей стилизованной графики. Эти идеи могут быть применены и в рамках рассматриваемого проекта, если в дальнейшем будет принято решение о дообучении Real-ESRGAN на соответствующих данных.

В отечественных исследованиях по архитектуре распределённых систем и веб-сервисов замечен интерес к языку программирования Golang и микросервисному подходу. В докладах и статьях описываются примеры

использования Golang для создания высоконагруженных API-шлюзов, систем логирования, сервисов доставки контента и подсистем аналитики. Отдельные публикации посвящены применению фреймворков, аналогичных Gin, для упрощения разработки REST-сервисов. Отмечается, что использование Golang позволяет получить компактные и производительные серверные компоненты, что важно для стриминговых платформ, где одновременно обслуживается множество пользователей.

Значительное внимание отечественные исследования уделяют системам обмена сообщениями и обработке потоков данных. Рассматривается использование Apache Kafka и подобных решений для построения конвейеров обработки событий, логирования, аналитики и интеграции микросервисов. В статьях описываются типовые паттерны применения Kafka: очереди задач, топики логов, шины данных для аналитических систем. Такой опыт напрямую применим к проектируемому сервису: события, связанные с загрузкой и обработкой видео, могут передаваться через Kafka, что обеспечивает асинхронное взаимодействие и снижает связанность компонентов.

Отдельные отечественные работы затрагивают вопросы оценки качества видеоконтента и пользовательского опыта. Проводятся исследования, направленные на выявление факторов, наиболее сильно влияющих на субъективную оценку качества: резкость изображения, наличие шумов, стабильность и непрерывность воспроизведения, частота смены качества при адаптивном стриминге. Делается вывод о том, что даже при ограниченном битрейте грамотное сочетание алгоритмов кодирования, методов постобработки и адаптивной доставки позволяет достичь приемлемого уровня восприятия. Это подтверждает целесообразность использования суперразрешения и других методов улучшения изображения именно в связке с продуманной стриминговой архитектурой.

В целом отечественные исследования формируют богатый контекст, в рамках которого рассматриваемый в данной работе стриминговый сервис ретроконтента может быть осмыслен не как абстрактный учебный пример, а как

потенциально практико-ориентированное решение. Учитывая наработки в области обработки изображений, построения микросервисных архитектур, использования открытых технологий и соблюдения требований законодательства, можно сформулировать реалистичные требования к системе и пути её дальнейшего развития.

1.3 Общая характеристика проблемы исследования

Синтез данных, полученных в результате анализа зарубежных и отечественных литературных источников, а также практики создания и эксплуатации стриминговых сервисов, позволяет сформулировать общую характеристику проблемы, лежащей в основе настоящего исследования. В наиболее общем виде проблема заключается в отсутствии специализированного программного решения, ориентированного на легальный показ ретрофильмов и ретроаниме с одновременным обеспечением высокого качества изображения на современных устройствах и рационального использования вычислительных и сетевых ресурсов.

С одной стороны, существует устойчивый пользовательский интерес к ретроконтенту. Классические фильмы и аниме рассматриваются как важная часть культурного наследия, пользователи проявляют интерес к их просмотру в удобном цифровом формате. С другой стороны, техническое качество этих материалов зачастую не соответствует современным стандартам: низкое разрешение, наличие шумов, артефактов сжатия, дефектов плёнки и других искажений. При прямом воспроизведении такого контента на больших 4К-экранах создаётся эффект «растянутого» изображения, что ухудшает восприятие и снижает мотивацию к просмотру, особенно у зрителей, привыкших к высокому качеству современных релизов.

В теории проблему деградации качества можно было бы решить за счёт традиционного ремастеринга, включающего ручную реставрацию, повторное

сканирование плёнки с высоким разрешением и сложную цветокоррекцию. Однако такой подход требует значительных финансовых и временных затрат, а потому оправдан только для наиболее коммерчески успешных и востребованных произведений. Для массовой библиотеки ретрофильмов и аниме такой путь оказывается мало реалистичным. В результате значительная часть контента остаётся в состоянии, не соответствующем ожиданиям зрителей.

Параллельно в научной и инженерной среде были разработаны методики суперразрешения и нейросетевой реконструкции изображений и видео, позволяющие автоматически повышать качество и разрешение материалов. Модели, подобные Real-ESRGAN, демонстрируют способность восстанавливать детали и улучшать визуальное восприятие даже при работе с сильно деградированными данными. Однако между существованием таких моделей и их массовым применением в реальных стриминговых сервисах существует заметный разрыв. Этот разрыв обусловлен не только вычислительной сложностью моделей, но и необходимостью продуманной архитектурной интеграции в конвейер подготовки и доставки контента.

Таким образом, проблема исследования проявляется в нескольких взаимосвязанных плоскостях:

- 1) **Техническая плоскость.** Необходимо обеспечить совместное функционирование стриминговой подсистемы и подсистемы нейросетевой обработки. Требуется разработать архитектуру, при которой сервис сможет:

хранить исходные и улучшенные версии видеоматериалов;

по возможности выполнять улучшение в фоновом режиме, не блокируя основную работу сервиса;

эффективно использовать доступные вычислительные ресурсы (CPU, GPU), избегая перегрузок и обеспечивая предсказуемое время обработки.

- 2) **Архитектурная плоскость.** Требуется определить оптимальный подход к организации компонентов системы: какие функции должны быть вынесены в отдельные микросервисы, как между ними распределить

ответственность и как организовать взаимодействие. В качестве ключевого архитектурного решения рассматривается использование брокера сообщений (Kafka) для асинхронной постановки задач на обработку видео и передачи событий между компонентами. Это предполагает переход от монолитных схем к более гибким событийно-ориентированным архитектурам.

- 3) **Данных и хранения.** Нужно спроектировать систему хранения, в которой метаданные (информация о пользователях, фильмах, статусах обработки) и крупные бинарные объекты (видео, постеры, сегменты) будут храниться в разных подсистемах (PostgreSQL и MinIO), но при этом оставаться логически связанными. Также необходимо учитывать, что хранение нескольких версий одного и того же материала (оригинал и 4K-upscaled) увеличивает требования к объёму хранилища и требует продуманных стратегий управления жизненным циклом данных.
- 4) **Пользовательская плоскость.** Важной частью проблемы является обеспечение комфортного пользовательского опыта. Пользователь не должен осознавать внутреннюю сложность системы; его интересуют доступность контента, удобство поиска, скорость запуска и качество воспроизведения. Задача разработчика — скрыть за простым и интуитивно понятным интерфейсом сложный конвейер обработки и доставки данных. Это требует аккуратного проектирования интерфейса и логики взаимодействия клиента с сервером, включая отображение статусов (например, «идёт обработка улучшенной версии»).
- 5) **Эксплуатационная и развёртываемость.** Система должна быть не только работоспособной на этапе прототипа, но и потенциально пригодной к эксплуатации и развитию. Это означает необходимость закладывать в архитектуру элементы сопровождаемости: логирование, мониторинг, удобную конфигурацию, возможность поэтапного обновления компонентов без остановки всего сервиса. Важно также учитывать

перспективу масштабирования при росте числа пользователей и объёма контента.

Обобщая вышеизложенное, можно сформулировать основную проблему исследования как отсутствие комплексного решения, которое:

учитывает специфику ретроконтента (низкое качество исходных материалов, разнообразие артефактов, отдельные особенности анимации);

сочетает современные методы нейросетевого upscaling (на базе Real-ESRGAN и PyTorch) с промышленными подходами к организации стриминга (HLS/DASH, адаптивный битрейт);

опирается на открытый и расширяемый стек технологий (Golang, Gin, PostgreSQL, MinIO, Kafka);

обеспечивает приемлемый баланс между качеством изображения, производительностью, масштабируемостью и удобством эксплуатации.

Решение этой проблемы требует системного подхода, включающего теоретический обзор, выбор и обоснование методов и технологий, формирование требований к системе и разработку архитектурных принципов её построения. Именно этим аспектам посвящены последующие разделы настоящей работы.

2. ОБЗОР ТЕХНОЛОГИЙ И МЕТОДОВ РЕШЕНИЯ ПРОБЛЕМЫ

2.1 Основные методы решения проблемы и их характеристики

Для решения обозначенной проблемы в современной практике применяются различные методы, которые можно условно разделить на несколько групп, каждая из которых закрывает свой аспект задачи.

Первая группа — методы организации потоковой передачи видеоданных. Наиболее распространён подход сегментированной доставки видео по протоколам HTTP с использованием адаптивного битрейта. В этом случае исходный видеопоток предварительно кодируется в несколько вариантов качества и битрейта, после чего на стороне клиента динамически выбирается наиболее подходящий вариант в зависимости от состояния сети и состояния буфера. Такой подход позволяет обеспечить приемлемое качество изображения даже при колебаниях пропускной способности канала, а также максимально эффективно использовать доступную полосу. С точки зрения реализации это требует дополнительного места в хранилище, так как для каждого видео хранится несколько профилей качества, однако преимущества по стабильности воспроизведения и удобству для пользователя делает этот метод де-факто стандартом индустрии.

В дополнение к классическому адаптивному стримингу в исследованиях рассматриваются методы оптимизации выбора битрейта, основанные на интеллектуальном анализе поведения сети и пользовательской активности. Например, алгоритмы могут учитывать не только текущую скорость загрузки сегментов, но и тип устройства, размер экрана и предпочтения пользователя, позволяя более точно подбирать уровень качества. Такие методы теоретически могут быть интегрированы в разрабатываемый сервис на последующих этапах

развития, однако в рамках данной работы достаточно предусмотреть базовую поддержку адаптивного потока.

Вторая группа — методы повышения качества и разрешения видеоконтента. Для простых сценариев применяются традиционные алгоритмы интерполяции (bilinear, bicubic и др.), которые обеспечивают быстрое масштабирование, но зачастую приводят к размытию и потере деталей, особенно на старых и сильно сжатых видеозаписях. Эти методы просты в реализации и практически не требуют специализированного оборудования, однако при работе с ретроконтентом их ограничений становится слишком много: усиливаются артефакты сжатия, контуры становятся менее чёткими, часть мелких деталей теряется безвозвратно.

В противоположность этому, методы суперразрешения на основе глубоких нейронных сетей обучаются на соответствиях между низкокачественными и высококачественными изображениями и поэтому способны восстанавливать недостающие элементы и текстуры. Модель Real-ESRGAN относится к классу генеративно-состязательных сетей (GAN-подход), которые позволяют получать визуально правдоподобное изображение за счёт конкуренции генератора и дискриминатора. Реализация модели на базе PyTorch даёт гибкость в настройке, дообучении и интеграции в различные программные среды. Основным недостатком такого подхода являются значительные вычислительные затраты, особенно при обработке длинных видеороликов и при попытке вывести модель в режим, близкий к реальному времени.

Третья группа методов относится к проектированию архитектуры самого сервиса. Здесь можно выделить монолитные и микросервисные подходы. Монолитные приложения предполагают объединение всей логики в один исполняемый модуль, что упрощает начальную реализацию и развертывание, но затрудняет масштабирование, обновление и введение новых функций без риска нарушения работы уже существующих. Микросервисная архитектура, наоборот, предполагает разделение системы на отдельные компоненты, каждый из которых отвечает за конкретный набор функций (каталог контента, хранение, стриминг,

авторизация, обработка видео, аналитика). Это облегчает независимое масштабирование отдельных подсистем, упрощает обновление и позволяет использовать разные технологические стеки внутри одной системы. Однако такой подход предъявляет более высокие требования к инфраструктуре, мониторингу, логированию и средствам интеграции.

Четвёртая группа методов связана с организацией асинхронного взаимодействия между компонентами системы. Для разграничения задач приёма пользовательских запросов и тяжёлых фоновых вычислений широко используются брокеры сообщений и шины данных, такие как Apache Kafka. Они позволяют строить событийно-ориентированные архитектуры, в которых операции загрузки видео, постановка задач на upscaling, обновление статуса обработки и уведомление пользователей выполняются через очереди и топики сообщений. Такой подход уменьшает связанность компонентов, повышает устойчивость к пиковым нагрузкам и упрощает отладку и развитие системы, поскольку каждый сервис может обрабатывать события в своём темпе, не блокируя остальные части системы.

Для рассматриваемого стримингового сервиса целесообразно сочетать перечисленные группы методов: использовать адаптивный стриминг для выдачи контента, нейросетевой upscaling для повышения качества ретровидео, микросервисную архитектуру для структурирования системы и Kafka для организации асинхронного взаимодействия между компонентами. Такое сочетание обеспечивает баланс между гибкостью, расширяемостью и практической реализуемостью решения в рамках учебного проекта.

2.2 Основные технологии решения проблемы и их характеристики

На технологическом уровне реализация стримингового сервиса с поддержкой 4K-upscaling ретроко́н­тента опирается на сочетание серверных,

хранилищных, вычислительных и интеграционных инструментов. Важным критерием выбора технологий является их открытость, активное развитие и наличие достаточной документации и сообщества, что особенно важно в контексте учебного проекта.

В качестве языка серверной части предлагается использовать Golang. Данный язык отличает высокая производительность, статическая типизация и развитая поддержка конкурентности (goroutines, channels), что делает его удобным для создания сетевых и веб-приложений. Благодаря встроенным средствам работы с HTTP, простоте организации многопоточности и относительной компактности исполняемых файлов Golang хорошо подходит для реализации высоконагруженных сервисов, отвечающих за раздачу видеосегментов, обработку запросов пользователей, взаимодействие с подсистемами хранения и базами данных. Кроме того, язык имеет богатую экосистему библиотек для интеграции с Kafka, PostgreSQL, MinIO и другими распространёнными компонентами современных распределённых систем.

В качестве базового веб-фреймворка для реализации REST-API и маршрутизации HTTP-запросов планируется использовать Gin router (Gin Web Framework) для языка Go. Gin предоставляет удобный и лаконичный интерфейс для описания маршрутов, работы с middleware, обработки JSON-запросов и ответов, а также интеграции с системами логирования и аутентификации. Благодаря высокой производительности и небольшому накладному коду Gin получил широкое распространение в промышленных проектах, ориентированных на создание API-шлюзов и микросервисов. В контексте разрабатываемого сервиса Gin-приложение будет выступать в роли «фасада», принимающего запросы от фронтенда и внешних клиентов, обрабатывающего их и перенаправляющего к внутренним сервисам.

Для управления структурированными данными (метаданные о фильмах и аниме, информация о пользователях, профили, списки «избранного», история просмотров) выбирается реляционная СУБД PostgreSQL. PostgreSQL обеспечивает поддержку транзакций, богатый язык запросов SQL, развитые

механизмы индексации и оптимизации, а также возможности репликации и резервного копирования. В рамках учебного проекта использование PostgreSQL позволяет отработать навыки проектирования схемы БД, составления запросов и взаимодействия с базой данных из кода на Golang, а в перспективе — легко масштабировать систему и расширять её функциональность.

Хранение самих видеоматериалов и производных файлов (сегментов, постеров, обложек, субтитров) целесообразно организовать в объектном хранилище MinIO, совместимом с API Amazon S3. MinIO позволяет создавать отказоустойчивые и масштабируемые кластеры для хранения больших объёмов медиаданных, поддерживает версионирование, политики доступа и интеграцию с внешними сервисами резервного копирования и мониторинга. Использование MinIO в связке с PostgreSQL означает, что крупные бинарные объекты (видео, изображения) хранятся в объектном хранилище, тогда как в БД содержатся только ссылки и метаданные. Это уменьшает нагрузку на БД, упрощает горизонтальное масштабирование и позволяет легко переносить систему между различными средами и конфигурациями.

Специфическим элементом технологического стека является модель Real-ESRGAN, позволяющая выполнять нейросетевое масштабирование изображений и видео. Модель реализована на базе фреймворка PyTorch, который предоставляет гибкие средства описания нейросетевых архитектур, управления обучением и оптимизацией, а также достаточно удобные инструменты для выполнения вывода (inference) на CPU и GPU. В контексте разрабатываемого сервиса PyTorch будет использован на стороне отдельного ML-сервиса, который принимает задания на upscaling (например, через Kafka), загружает исходные кадры или фрагменты видео из MinIO, выполняет обработку и сохраняет результат обратно в хранилище. Такой подход позволяет отделить ресурсоёмкие операции от основного веб-сервиса, избегая ситуации, когда долгие вычисления блокируют реакции на пользовательские запросы.

Для интеграции компонентов и организации асинхронного конвейера обработки предлагается использовать систему обмена сообщениями Apache

Kafka. Kafka обеспечивает высокопроизводительный приём и доставку потоков событий, устойчиво работает при больших объёмах данных и поддерживает горизонтальное масштабирование. В рассматриваемом решении Kafka может использоваться для постановки задач на обработку видео (создание сообщений при загрузке нового файла или при запросе пользователем улучшенной версии), обновления статусов обработки (например, «обработка начата», «обработка завершена», «возникла ошибка»), а также сбора событий пользовательской активности (просмотры, паузы, перемотки) для последующей аналитики.

Дополнительно в технологический стек входят инструменты для кодирования и трансформации видео, среди которых наиболее распространённым является `ffmpeg`. С его помощью можно выполнять конвертацию форматов, нарезку видео на сегменты для HLS или MPEG-DASH, а также извлекать отдельные кадры или звуковые дорожки для дальнейшей обработки. `ffmpeg` будет использоваться как часть конвейера предобработки: после загрузки исходного файла запускается набор утилит, которые создают различные профили качества и формируют структуру каталогов в MinIO, соответствующую требованиям адаптивного стриминга.

Важным элементом решения является фронтенд, обеспечивающий взаимодействие пользователя с системой. В рамках данной работы акцент делается на серверной части, однако предполагается наличие минимального веб-интерфейса, реализованного с использованием стандартных веб-технологий (HTML, CSS, JavaScript) и взаимодействующего с сервером по REST-API. Такой подход позволяет сосредоточиться на архитектуре сервера и конвейере обработки, не углубляясь в сложные фронтенд-фреймворки, что соответствует целям выпускной квалификационной работы.

В совокупности указанные технологии формируют основу для построения прототипа стримингового сервиса, удовлетворяющего поставленным требованиям и демонстрирующего современный подход к архитектуре мультимедийных систем: использование микросервисов, событийно-ориентированного взаимодействия и нейросетевых методов обработки данных.

2.3 Алгоритм выбора методов и технологий решения проблемы

На основании проведённого анализа предлагается сформировать алгоритм выбора методов и технологий решения задачи разработки стримингового сервиса ретроконента с 4K-upscaling. Этот алгоритм описывает логическую последовательность шагов, которые необходимо выполнить при проектировании системы, и обосновывает выбор ключевых архитектурных решений и компонентов стека.

- 1) **Анализ предметной области и требований пользователей.** На данном этапе формулируется видение конечного продукта: для кого создаётся сервис, какие сценарии использования будут наиболее характерны, какой тип контента планируется размещать (классические фильмы, аниме, возможно, сериалы). Выделяются ключевые сценарии использования: просмотр ретрофильмов и аниме в браузере, выбор качества видео, возможность переключения между оригинальной и улучшенной версиями, поиск и фильтрация по каталогу, сохранение «избранного», продолжение просмотра с места остановки. На этом шаге также формируются первичные ожидания по качеству изображения (до 4K), допустимым задержкам, количеству одновременных пользователей и допустимому времени обработки новых загрузок.
- 2) **Определение ограничений инфраструктуры.** Здесь оцениваются доступные вычислительные ресурсы (количество CPU-ядер, наличие и тип GPU, объём оперативной памяти), объёмы хранилища (общий и по категориям «холодного» и «горячего» хранения), пропускная способность сети (внутри сервера, между сервисами, во внешние сети) и предполагаемое количество одновременных пользователей. От этого зависит, можно ли позволить себе полностью онлайн-upscaling по запросу

пользователя или целесообразнее придерживаться офлайн-модели с предварительной обработкой наиболее востребованного контента. Для учебного проекта обычно предполагается ограниченный набор ресурсов, поэтому выбор склоняется в сторону гибридного подхода с преобладанием офлайн-обработки.

3) Сопоставление требований с характеристиками методов стриминга.

На данном шаге сравниваются различные протоколы и форматы стриминга (прямой прогрессивный поток, HLS, MPEG-DASH, использование pseudo-streaming) с точки зрения совместимости с браузерами, сложностью реализации и возможностями адаптации качества. С учётом широкого распространения и поддержки на стороне клиентов выбирается вариант сегментированной доставки видео по HTTP с адаптивной подстройкой битрейта и разрешения. Параллельно определяется формат хранения сегментов в MinIO и структура каталогов.

4) Выбор методов повышения качества.

После определения базового подхода к стримингу выбираются методы повышения качества контента. Сравниваются классические масштабирующие алгоритмы и нейросетевые методы суперразрешения. Учитывая требование к высокому визуальному качеству ретроконтента и желание продемонстрировать современные техники компьютерного зрения, принимается решение о применении Real-ESRGAN в качестве основного инструмента super-resolution, а классические алгоритмы рассматриваются как резервный вариант на случай недоступности ML-сервиса или ограничения ресурсов. На этом шаге также принимается решение в пользу фреймворка PyTorch как платформы для развёртывания модели.

5) Формирование технологического стека.

На основе требований к производительности, надёжности, удобству разработки и доступности документации формируется окончательный стек технологий: Golang и Gin для серверной логики и REST-API, PostgreSQL для управления метаданными, MinIO для объектного хранения медиаданных, PyTorch и

Real-ESRGAN для нейросетевого upscaling, Kafka для событийно-ориентированного взаимодействия между сервисами, ffmpeg для кодирования и сегментации видео. На этом шаге проверяется совместимость выбранных компонентов и прорабатываются базовые сценарии их интеграции.

- 6) **Проектирование архитектуры и потоков данных.** Определяется набор микросервисов, их роли, ответственности и точки интеграции. Выделяется сервис приёма пользовательских запросов (на Golang и Gin), сервис управления каталогом и метаданными, сервис загрузки и подготовки контента, ML-сервис на PyTorch, выполняющий upscaling, а также сервисы мониторинга, логирования и административного интерфейса. Описываются потоки данных: от загрузки исходного видео в MinIO до публикации сегментов для стриминга после обработки Real-ESRGAN. Определяется, какие события генерируются и обрабатываются через Kafka, какие данные хранятся в PostgreSQL и какие — в MinIO.
- 7) **Оценка интеграционных и эксплуатационных аспектов.** Анализируются сложности развертывания и администрирования выбранных технологий, наличие готовых контейнеров и helm-чартов, возможности для автоматизации развёртывания и обновления. Определяются базовые подходы к логированию, мониторингу и обработке ошибок. На этом шаге также оценивается трудоёмкость разработки и сопровождения системы в дальнейшем, что важно в контексте ВКР: архитектура должна быть не только теоретически обоснованной, но и реалистичной для практической реализации в установленные сроки.
- 8) **Поэтапная реализация и план развития.** Заключительный шаг алгоритма заключается в формировании поэтапного плана реализации: от создания минимально жизнеспособного прототипа (MVP) до постепенного добавления дополнительных функций и оптимизаций. На первом этапе может быть реализован минимальный набор функций (загрузка видео, каталог, базовый стриминг, упрощённый офлайн-upscaling), после чего по

мере готовности добавляются Kafka-интеграция, расширенный интерфейс и дополнительные нефункциональные улучшения. Такой поэтапный подход снижает риски и позволяет наглядно демонстрировать прогресс по мере выполнения ВКР.

Финальный выбор методов и технологий осуществляется с учётом совокупной эффективности, стоимости владения и возможности поэтапной реализации в рамках выпускной квалификационной работы, начиная с прототипа и постепенно расширяя функциональность и масштаб.

3 ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К РЕШЕНИЮ ПРОБЛЕМЫ

3.1 Общая характеристика предлагаемого решения

В качестве предлагаемого решения рассматривается программный комплекс — веб-ориентированный стриминговый сервис, предназначенный для хранения, обработки и потоковой передачи ретрофильмов и ретроаниме с возможностью просмотра в разрешении до 4К. Сервис должен обеспечивать пользователям доступ к каталогу ретроконтента через браузерный интерфейс, предоставлять информацию о произведениях и позволять воспроизводить видео с учётом выбранного качества и текущих сетевых условий. Особенностью решения является наличие подсистемы нейросетевого upscaling, основанной на модели Real-ESRGAN и фреймворке PyTorch, а также использование Kafka для организации асинхронной обработки задач.

С точки зрения пользователя сервис должен выглядеть как привычный онлайн-кинотеатр: после авторизации пользователь видит каталог доступных ретрофильмов и аниме, может просматривать карточки произведений, выбирать интересующий контент, запускать воспроизведение и, при наличии, переключаться между исходной и улучшенной версиями видео. Дополнительно система может предлагать базовые возможности персонализации — например, список «избранного» и сохранение позиции остановки просмотра.

Архитектурно решение предполагается построить по модульному принципу. В его состав входят следующие основные компоненты:

- 1) модуль управления пользователями и аутентификации, отвечающий за регистрацию, авторизацию, хранение профилей и разграничение прав доступа;
- 2) модуль каталога и поиска по ретроконтенту, реализующий функции добавления, просмотра, поиска и фильтрации записей о видеоматериале;

- 3) модуль хранения метаданных на базе PostgreSQL, обеспечивающий надёжное хранение структурированной информации о пользователях и контенте;
- 4) модуль объектного хранения видеоматериалов на базе MinIO, организующий физическое размещение исходных и улучшенных видеороликов, а также связанных файлов;
- 5) модуль подготовки и трансформации видео (кодирование, сегментация, upscaling с использованием Real-ESRGAN), включающий в себя утилиты ffmpeg и ML-сервис на PyTorch;
- 6) модуль стриминга, отвечающий за выдачу видеосегментов пользователям в соответствии с выбранным протоколом адаптивного стриминга;
- 7) модуль обмена сообщениями на базе Kafka, обеспечивающий асинхронное взаимодействие между сервисами загрузки, обработки и обновления каталога;
- 8) минимальный фронтенд для взаимодействия пользователя с системой (просмотр каталога, выбор видео, управление воспроизведением, авторизация).

Веб-API и основная часть серверной логики предлагается реализовать на языке Golang с использованием фреймворка Gin router. Такой выбор обусловлен высокой производительностью Golang, удобством работы с HTTP и хорошей интеграцией с PostgreSQL, MinIO и Kafka. Gin обеспечивает удобную маршрутизацию запросов, работу с middleware и простую сериализацию/десериализацию JSON, что упрощает реализацию REST-интерфейса.

ML-обработка будет вынесена в отдельный сервис, реализованный на базе PyTorch. Этот сервис принимает задания из Kafka, загружает исходные данные из MinIO, выполняет upscaling с использованием Real-ESRGAN и сохраняет результат обратно в хранилище, после чего посылает уведомление о завершении обработки. Такое разнесение функциональности позволяет независимо масштабировать веб-часть и подсистему нейросетевой обработки, а также не

блокировать пользовательские запросы при выполнении ресурсоёмких операций.

Ожидается, что данное решение позволит придать ретроконтенту новое визуальное качество, сделать его более привлекательным для современных зрителей и одновременно продемонстрировать практическую применимость открытых технологий (Golang, Gin, PostgreSQL, MinIO, PyTorch, Kafka, Real-ESRGAN) для реализации стриминговых платформ. Модульная архитектура упростит дальнейшее расширение системы, например, добавление рекомендательной системы, поддержки мобильных клиентов, интеграции с социальными сетями или подключения внешних сервисов аналитики.

3.2 Функциональные требования к предлагаемому решению

Функциональные требования определяют, что именно должна делать система и какие сценарии использования она обязана поддерживать. Для разрабатываемого стримингового сервиса можно выделить следующие ключевые группы функциональных требований.

1) Управление

пользователями

- a. регистрация пользователей с указанием минимального набора данных (логин, пароль, при необходимости — контактный e-mail);
- b. авторизация пользователей с проверкой введённых учётных данных и предоставлением доступа к персонализированным функциям (история просмотров, «избранное»);
- c. хранение профилей пользователей, включая базовую информацию и настройки предпочтений (например, предпочтительное качество видео, язык интерфейса);
- d. разграничение прав доступа (например, гость, зарегистрированный пользователь, администратор), позволяющее ограничивать доступ к административным функциям и мониторингу;

е. возможность сброса пароля и восстановления доступа к учётной записи с использованием стандартных механизмов (через e-mail или другие средства);

ф. ведение истории активности пользователя (просмотренные фильмы, время последнего входа и т. д.) для последующей аналитики и улучшения сервиса.

2) Работа с каталогом контента

а. хранение информации о ретрофильмах и аниме (название, оригинальное название, год выпуска, страна, жанр, короткое описание, постер, длительность, формат);

б. возможность поиска по названию и фильтрации по основным параметрам (год, жанр, страна, тип контента — фильм или сериал, наличие улучшенной версии);

с. отображение карточки произведения с подробной информацией, включая технические параметры доступных версий (исходная, 4K-upscaled), и, при необходимости, юридическую информацию;

д. поддержка пагинации и сортировки результатов (по популярности, дате добавления, году выпуска, алфавиту);

е. возможность помечать контент как «избранный» и отображать соответствующие списки для авторизованных пользователей;

ф. предоставление административных функций по добавлению нового контента, редактированию уже существующих записей и удалению устаревших или ошибочных записей.

3) Хранение и подготовка видеоданных

а. загрузка исходных видеоматериалов через административный интерфейс или служебный API с последующим размещением файлов в MinIO;

б. автоматическое создание записей в PostgreSQL при загрузке нового контента, включающих базовые метаданные и ссылки на файлы в хранилище;

с. инициирование процессов предобработки: кодирование исходного видео

в несколько вариантов качества и битрейта (например, 720p, 1080p, 4K), нарезка на сегменты для адаптивного стриминга;

d. постановка задач на нейросетевой upscaling с использованием Kafka (генерация сообщений о необходимости обработки конкретного видео или его части);

e. выполнение upscaling ретровидео до 4K с использованием модели Real-ESRGAN в отдельном ML-сервисе на PyTorch;

f. сохранение улучшенных версий видео и их сегментов в MinIO с обновлением записей в PostgreSQL (указание доступности 4K-версии и других параметров).

4) Потокковая передача видео

a. организация выдачи видеосегментов по протоколу HTTP на основе выбранного формата адаптивного стриминга (например, HLS);

b. предоставление плейлистов и метаданных для медиа-плеера, позволяющих клиенту динамически выбирать качество в зависимости от доступной пропускной способности сети;

c. поддержка корректного возобновления воспроизведения при временных сбоях связи, включая повторную загрузку сегментов и восстановление позиции;

d. обработка запросов от игрока (пауза, перемотка, смена качества) и генерация соответствующих событий, которые могут быть далее использованы в аналитике;

e. обеспечение совместимости с основными современными браузерами и встроенными плеерами.

5) Пользовательский интерфейс

a. отображение списка доступного контента с возможностью фильтрации и сортировки;

b. предоставление пользователю удобной формы поиска по названию и другим параметрам;

c. показ детальной карточки фильма или аниме с описанием, постером,

техническими характеристиками и информацией о доступности
улучшенной версии;

d. запуск воспроизведения в встроенном плеере, позволяющем управлять основными функциями (пауза, воспроизведение, перемотка, выбор качества);

e. возможность перехода между оригинальной и улучшенной (4K-upscaled) версией при их одновременной доступности, с отображением наглядных индикаторов, какая версия выбрана;

f. доступ к личному кабинету, где пользователь может просматривать историю просмотров, управлять списком «избранного» и изменять отдельные настройки.

6) Администрирование

a. доступ к административной панели только для пользователей с соответствующей ролью;

b. добавление, редактирование и удаление записей о контенте, включая загрузку постеров, корректировку описаний и изменение статусов;

c. управление процессами загрузки и обработки видео: запуск, остановка, просмотр текущего состояния очереди задач в Kafka;

d. просмотр базовой статистики по просмотрам, загрузкам и активности пользователей в агрегированном виде;

e. управление учётными записями пользователей (изменение ролей, блокировка при нарушении правил использования сервиса).

7) Интеграция и внутренние сервисы

a. предоставление REST-API, реализованного на Golang и Gin, для взаимодействия фронтенда и потенциальных внешних клиентов (например, мобильных приложений);

b. интеграция с Kafka для обмена событиями между сервисами загрузки, обработки видео и обновления каталога;

c. предоставление служебных API для контроля состояния ML-сервиса (проверка доступности, версии модели, загрузки ресурсов);

d. возможность интеграции с системами мониторинга и логирования для анализа метрик и поведения приложения.

Каждое функциональное требование должно быть сформулировано таким образом, чтобы быть конкретным, реализуемым в рамках выбранного стека технологий и поддающимся проверке в процессе тестирования прототипа и последующей эксплуатации.

3.3 Нефункциональные требования к предлагаемому решению

Нефункциональные требования описывают свойства системы, не связанные напрямую с реализуемыми функциями, но определяющие качество её работы, удобство использования и сопровождения.

1) Производительность

- a. время отклика сервера на типовые запросы (получение информации о фильме, авторизация, старт воспроизведения) не должно превышать разумного порога, составляющего, как правило, 1–2 секунды при нормальной нагрузке;
- b. задержка начала воспроизведения видео после запроса пользователя должна быть минимальной; пользователь не должен длительно ожидать загрузки первого сегмента;
- c. подсистема нейросетевой обработки на PyTorch должна обеспечивать приемлемое время обработки видеороликов с учётом доступных ресурсов, при этом нагрузка на систему должна сглаживаться за счёт использования очередей Kafka;
- d. использование Golang и Gin на стороне серверной логики должно позволять обрабатывать параллельно множество запросов без значительного роста времени ответа.

2) Масштабируемость

- a. архитектура системы должна позволять увеличивать количество экземпляров серверных компонентов (Golang-сервисов) и узлов MinIO по мере роста числа пользователей и объёма контента;
- b. должна быть предусмотрена возможность выделения отдельных сервисов (стриминг, обработка видео, служебные API, ML-сервис на PyTorch) в самостоятельные компоненты, масштабируемые независимо;
- c. Kafka-кластер должен поддерживать увеличение числа брокеров и разделов (partitions) для повышения пропускной способности и устойчивости к нагрузкам;
- d. выбор технологий и способов деплоя должен учитывать возможность перехода от единичной инсталляции к распределённому развертыванию в контейнерах или облачной среде.

3) Надёжность и отказоустойчивость

- a. система должна быть устойчива к сбоям отдельных компонентов: отказ одного из сервисов не должен приводить к полной недоступности сервиса для пользователей;
- b. данные о пользователях, метаданных контента и статусах обработки должны быть защищены от потери за счёт использования механизмов резервного копирования, репликации и журналирования;
- c. временные сбои в работе ML-сервиса или Kafka не должны блокировать возможность просмотра уже обработанного контента; соответствующие ошибки должны фиксироваться и обрабатываться с возможностью повторной попытки;
- d. архитектура должна поддерживать восстановление после сбоев без необходимости полной перезагрузки системы и с минимальным участием администратора.

4) Безопасность

- a. система должна реализовывать механизмы аутентификации и авторизации пользователей, используя проверенные и устойчивые к

типичным атакам подходы (хранение паролей в хэшированном виде, защита от SQL-инъекций, XSS и т. д.);

b. взаимодействие между клиентом и сервером должно осуществляться по защищённому протоколу (HTTPS), исключающему возможность перехвата и подмены данных;

c. доступ к административным функциям должен быть строго ограничен, все действия администраторов должны записываться в журнал;

d. необходимо учитывать требования по защите персональных данных и авторских прав, реализуя соответствующие ограничения на доступ к контенту и данным пользователей;

e. внутренняя коммуникация между сервисами (в том числе с Kafka и ML-сервисом) должна быть защищена от несанкционированного доступа с использованием аутентификации и настроек доступа.

5) Удобство

использования

a. пользовательский интерфейс должен быть интуитивно понятным, логика навигации — очевидной; основные действия (поиск фильма, запуск просмотра) должны быть доступны в минимум шагов;

b. интерфейс должен корректно отображаться в современных браузерах и на различных устройствах, адаптируясь к размерам экрана;

c. система должна предоставлять понятные сообщения об ошибках и подсказки, позволяющие пользователю ориентироваться в случае проблем;

d. на страницах, где требуется ожидание (например, при запуске улучшенной версии видео, если обработка выполняется по запросу), должны отображаться индикаторы прогресса или статуса.

6) Сопровождаемость

и

расширяемость

a. исходный код серверной части на Golang должен быть структурирован по модулям, разделён на логические слои (маршрутизация, бизнес-логика, доступ к данным), что облегчит его поддержку и развитие;

b. конфигурация системы (адреса сервисов, параметры подключения к БД, настройки Kafka, параметры моделей PyTorch) должна быть вынесена в

конфигурационные файлы или переменные окружения, позволяя изменять их без перекомпиляции приложений;

с. должна быть обеспечена возможность добавления новых типов контента или функций (например, рейтинги, комментарии, рекомендации, уведомления) без радикального изменения архитектуры;

d. система должна поддерживать внедрение дополнительных инструментов мониторинга и логирования, что позволит по мере необходимости расширять глубину анализа поведения приложения и выявлять узкие места.

Указанные нефункциональные требования задают рамки, в пределах которых планируется реализация стримингового сервиса и последующая оценка его качества. Они служат ориентиром при выборе архитектурных решений, технологий и компромиссов между производительностью, надёжностью, безопасностью и удобством использования.

ЗАКЛЮЧЕНИЕ

В рамках выполнения научно-исследовательской работы был проведён комплексный анализ проблемной области, связанной с разработкой стримингового сервиса для показа ретрофильмов и ретроаниме с возможностью потоковой передачи видео в разрешении до 4K. Рассмотрены особенности существующих стриминговых платформ, подходы к организации видеостриминга, методы повышения качества изображения и архитектурные принципы построения современных веб-сервисов, ориентированных на высокую нагрузку и отказоустойчивость.

Отдельное внимание уделено применению нейросетевых моделей суперразрешения, в частности Real-ESRGAN, реализованной на базе фреймворка PyTorch. Показано, что использование подобных моделей позволяет существенно улучшить визуальное качество ретроконтента, который в исходном виде представлен в низком разрешении и содержит различные артефакты. Обоснована необходимость интеграции нейросетевой обработки в архитектуру стримингового сервиса, что требует продуманного разделения ответственности между компонентами и использования асинхронных механизмов взаимодействия.

В результате исследования были систематизированы и критически оценены методы и технологии, применяемые для решения задач хранения, обработки и доставки видеоконтента. Особое внимание уделено выбору технологического стека, включающего Golang и фреймворк Gin для реализации серверной логики и REST-API, PostgreSQL для управления метаданными, MinIO для объектного хранения медиаданных, PyTorch и Real-ESRGAN для нейросетевого upscaling, а также Kafka для организации асинхронного обмена сообщениями и построения событийно-ориентированного конвейера обработки. Показано, что такой стек позволяет сочетать открытость, производительность, гибкость и возможность поэтапного развития системы.

На основе проведённого анализа был предложен алгоритм выбора методов и технологий, учитывающий требования к качеству изображения, производительности, масштабируемости и безопасности. Сформулирован комплекс функциональных и нефункциональных требований к стриминговому сервису, определяющий целевой облик системы и служащий ориентиром для последующего проектирования и реализации. Описана общая архитектура предлагаемого решения, включающая модули управления пользователями, каталогом контента, хранилищем данных, конвейером обработки видео, системой стриминга и фронтенд-компонентами.

Проведённое исследование создаёт прочный теоретический и методологический задел для перехода к практической стадии — непосредственной разработке прототипа стримингового сервиса ретроконтента с 4K-upscaling в рамках выпускной квалификационной работы. На этапе реализации планируется разработать схему данных в PostgreSQL, построить REST-API на Golang и Gin, интегрировать MinIO для хранения видеофайлов, настроить Kafka для обмена сообщениями между сервисами, а также внедрить модель Real-ESRGAN на базе PyTorch в конвейер обработки видеоконтента.

Дальнейшая деятельность может быть связана с оптимизацией производительности и качественных характеристик системы: уточнением параметров нейросетевой модели, экспериментами с различными стратегиями предобработки и кэширования контента, внедрением более продвинутых алгоритмов адаптивного стриминга, а также расширением функциональности пользовательского интерфейса и возможностей персонализации. Кроме того, представляется перспективным направление интеграции разработанного решения с внешними источниками данных и сервисами аналитики, что позволит на следующем этапе развития системы строить рекомендательные механизмы и более эффективно управлять каталогом ретроконтента.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data // arXiv. URL: <https://arxiv.org/abs/2107.10833> (дата обращения: 22.11.2025).
- [2] Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data // ICCV 2021 Workshops: The IEEE/CVF Conference on Computer Vision and Pattern Recognition. URL: https://openaccess.thecvf.com/content/ICCV2021W/AIM/papers/Wang_Real-ESRGAN_Training_Real-World_Blind_Super-Resolution_With_Pure_Synthetic_Data_ICCVW_2021_paper.pdf (дата обращения: 13.09.2025).
- [3] Real-ESRGAN Practical Algorithms for General Image/Video Restoration // GitHub. URL: <https://github.com/xinntao/Real-ESRGAN> (дата обращения: 06.12.2025).
- [4] Real-ESRGAN PyTorch Implementation // GitHub. URL: <https://github.com/ai-forever/Real-ESRGAN> (дата обращения: 06.12.2025).
- [5] Dynamic Adaptive Streaming over HTTP (MPEG-DASH) // Wikipedia. URL: https://en.wikipedia.org/wiki/Dynamic_Adaptive_Streaming_over_HTTP (дата обращения: 07.12.2025).
- [6] ISO/IEC 23009-1:2022 Information technology Dynamic adaptive streaming over HTTP (DASH) Part 1: Media presentation description and segment formats // ISO. URL: <https://www.iso.org/standard/83314.html> (дата обращения: 08.12.2025).
- [7] MPEG-DASH (ISO/IEC 23009) Dynamic Adaptive Streaming over HTTP // MPEG: Standards. URL: <https://www.mpeg.org/standards/> (дата обращения: 09.12.2025).
- [8] RFC 8216: HTTP Live Streaming // RFC Editor. URL: <https://www.rfc-editor.org/rfc/rfc8216> (дата обращения: 06.12.2025).

- [9] HTTP Live Streaming // Wikipedia. URL: https://en.wikipedia.org/wiki/HTTP_Live_Streaming (дата обращения: 11.12.2025).
- [10] HLS Stream M3U8: The Complete Guide for Developers // VideoSDK Developer Hub. URL: <https://www.videosdk.live/developer-hub/hls/hls-stream-m3u8> (дата обращения: 06.12.2025).
- [11] StreamingGuide Streaming with FFmpeg // FFmpeg Wiki. URL: <https://trac.ffmpeg.org/wiki/StreamingGuide> (дата обращения: 06.12.2025).
- [12] FFmpeg A complete, cross-platform solution to record, convert and stream audio and video // FFmpeg. URL: <https://ffmpeg.org/> (дата обращения: 03.11.2025).
- [13] Documentation FFmpeg Project // FFmpeg. URL: <https://ffmpeg.org/documentation.html> (дата обращения: 06.12.2025).
- [14] Streaming with FFmpeg: RTMP Basics, HLS/DASH Packaging // FFmpeg API Blog. URL: <https://www.ffmpeg.media/articles/streaming-rtmp-hls-dash> (дата обращения: 06.12.2025).
- [15] HLS Packaging using FFmpeg Easy Step-by-Step Tutorial // Ottverse. URL: <https://ottverse.com/hls-packaging-using-ffmpeg-live-vod/> (дата обращения: 01.11.2025).
- [16] How to design a Live Video Streaming System Like ESPN // GeeksforGeeks: System Design. URL: <https://www.geeksforgeeks.org/system-design/how-to-design-a-live-video-streaming-system-like-espn/> (дата обращения: 12.11.2025).
- [17] Design a Video Streaming Platform Like YouTube // Hello Interview: System Design. URL: <https://www.hellointerview.com/learn/system-design/problem-breakdowns/youtube> (дата обращения: 06.12.2025).
- [18] Documentation The Go Programming Language // Go.dev. URL: <https://go.dev/doc/> (дата обращения: 13.11.2025).
- [19] The Go Programming Language Official Site // Go.dev. URL: <https://go.dev/> (дата обращения: 13.11.2025).

- [20] Tutorials The Go Programming Language // Go.dev: Tutorials. URL: <https://go.dev/doc/tutorial/> (дата обращения: 02.11.2025).
- [21] Gin Web Framework High-performance HTTP web framework for Go // Gin-gonic. URL: <https://gin-gonic.com/> (дата обращения: 06.12.2025).
- [22] Documentation | Gin Web Framework // Gin-gonic. URL: <https://gin-gonic.com/en/docs/> (дата обращения: 06.12.2025). [Gin Web Framework+1](#)
- [23] Gin HTTP web framework written in Go (Golang) // GitHub. URL: <https://github.com/gin-gonic/gin> (дата обращения: 21.11.2025). [GitHub](#)
- [24] PostgreSQL: Documentation Home // PostgreSQL. URL: <https://www.postgresql.org/docs/> (дата обращения: 22.11.2025). [PostgreSQL](#)
- [25] PostgreSQL 18.1 Documentation // PostgreSQL. URL: <https://www.postgresql.org/docs/current/index.html> (дата обращения: 06.12.2025). [PostgreSQL](#)
- [26] PostgreSQL 18.0 Documentation // Postgres Professional. URL: <https://postgrespro.com/docs/postgresql/current/index> (дата обращения: 13.11.2025). [postgrespro.com](#)
- [27] MinIO High Performance Object Storage Documentation // MinIO Object Storage (AGPLv3). URL: <https://miniodocs.cc/> (дата обращения: 06.12.2025). [miniodocs.cc](#)
- [28] AISTor Object Store Documentation MinIO Enterprise // MinIO. URL: <https://docs.min.io/enterprise/aistor-object-store/> (дата обращения: 09.11.2025). [AISTor Object Store Documentation](#)
- [29] MinIO High Performance, S3 Compatible Object Storage // GitHub. URL: <https://github.com/minio/minio> (дата обращения: 06.12.2025). [GitHub](#)
- [30] Introduction to MinIO // Baeldung. URL: <https://www.baeldung.com/minio> (дата обращения: 09.11.2025). [Baeldung on Kotlin](#)
- [31] MinIO Object Storage // Wikipedia. URL: <https://en.wikipedia.org/wiki/MinIO> (дата обращения: 09.11.2025). [Википедия](#)